Array vs Linked List

데이터를 순서대로 저장해도 메모리에 놓이는 방식은 다르다.

ARRAY 배열

원소를 연속된 공간에 저장한다.

index 0	index 1	index 2	index 3
10	20	30	40
1000	1004	1008	1012

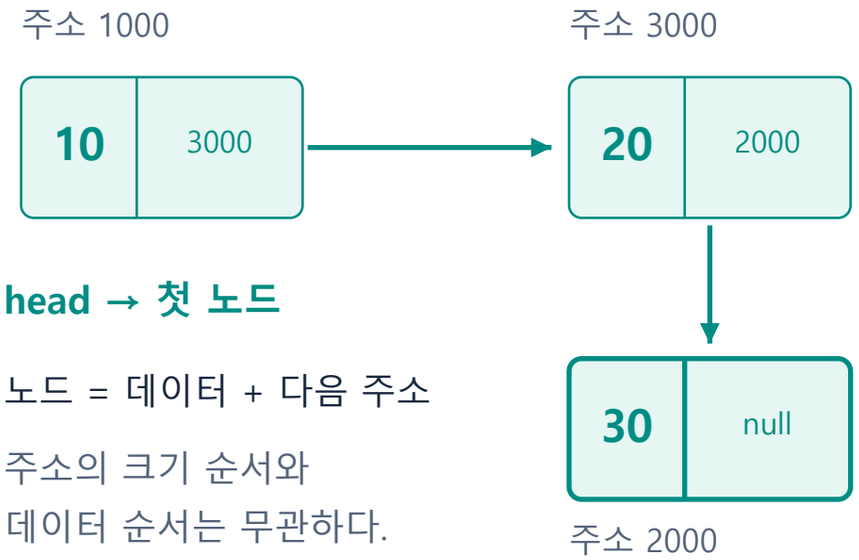
주소 = 시작 주소 + 인덱스 × 원소 크기

예: 4바이트 정수 → 주소가 4씩 증가

연속되는 것은 가상 메모리 주소 공간이다.

LINKED LIST 연결 리스트

노드를 따로 저장하고 주소로 연결한다.



head → 첫 노드

노드 = 데이터 + 다음 주소

주소의 크기 순서와
데이터 순서는 무관하다.

배열은 주소를 계산한다. 연결 리스트는 주소를 따라간다.

세 번째 원소를 읽으려면?

인덱스 접근: 배열은 바로 이동하고, 연결 리스트는 앞에서부터 따라간다.

ARRAY

인덱스 접근 O(1)

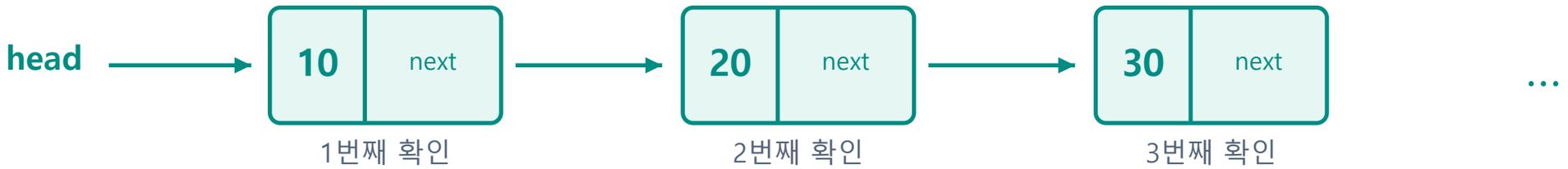


1000 + 2 × 4

= 주소 1008

LINKED LIST

인덱스 접근 O(n)



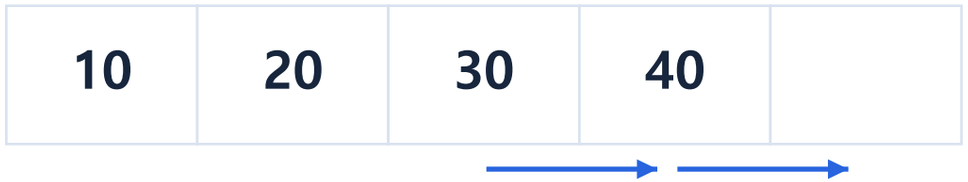
캐시 효율 배열은 이웃 원소도 함께 읽히기 쉬워 순차 처리에 대체로 유리하다.

20과 30 사이에 25를 넣어보자

배열은 원소를 이동한다. 연결 리스트는 연결을 바꾼다.

ARRAY

① 빈 공간 확보 후 30, 40을 뒤로 이동



② 빈 자리에 25 저장



뒤쪽 원소 이동 → O(n)

LINKED LIST

① 새 노드가 30을 가리키게 설정



② 20의 다음 주소를 새 노드로 변경

20 노드를 이미 알면 → O(1)

주의: 삽입 위치를 찾는 탐색이 필요하면 연결 리스트도 전체 O(n)

선택 기준은 “어떤 작업을 자주 하는가”

단일 연결 리스트 기준. n은 저장한 원소의 개수다.

비교 기준	Array	Linked List
메모리 배치	연속된 공간	노드별 배치 + 연결 주소
인덱스로 접근	O(1)	O(n)
값 검색 · 정렬 안 됨	O(n)	O(n)
중간 삽입·삭제	O(n) · 뒤 원소 이동	O(1)* · 연결 변경
메모리·캐시	주소 부담 없음 · 캐시 유리	주소·노드 관리 공간 필요
크기 변경	고정 배열: 크기 고정	노드를 추가·제거

* 연결 리스트는 필요한 노드를 알고 있을 때. 삭제는 보통 이전 노드도 알아야 한다.

동적 배열: 공간이 부족하면 더 큰 배열을 할당하고 복사한다. 예: Java ArrayList, Python list

같이 확인해 보기

Q. 100번째 원소를 자주 읽는다면? → 배열 | Q. 연결 리스트의 삽입은 항상 O(1)? → 아니오

O(1)은 “정확히 한 번”이라는 뜻이 아니다

데이터 개수 N이 늘어날 때, 작업량이 어떤 형태로 증가하는지를 본다.

증가하는 규모로 분류하기

2번, 100번처럼 고정	O(1)
3N, 100N, N/2번	O(N)
$2N^2 + 3N$번	$O(N^2)$

직선의 기울기가 달라도 모두 직선이다.

3N과 100N은 모두 O(N)이지만,
실제 실행 시간이 같다는 뜻은 아니다.

순서대로 하는 작업은 더한다

$$O(1) + O(1) = O(1)$$
$$O(1) + O(2) = O(1)$$
$$O(N) + O(1) = O(N)$$
$$O(N) + O(N) = O(N)$$

O(2)도 고정된 작업량이다.

따라서 보통 O(1)로 표현한다.

N과 n은 모두 원소 수를 나타내는 표기다.

고정된 100번 → O(1) | O(1) 작업을 N번 반복 → O(N)

20을 찾아서, 그 뒤에 25를 넣는다면?

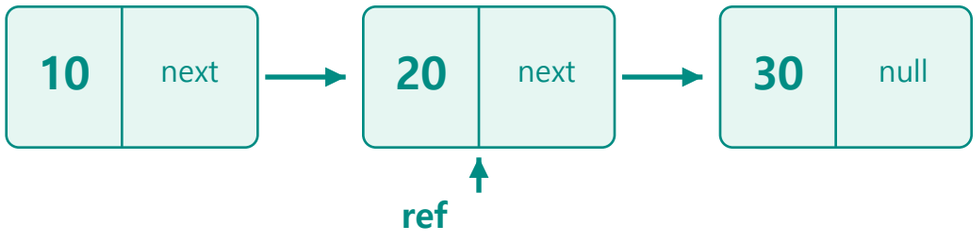
정렬되지 않은 배열과 단일 연결 리스트 비교. 시간 복잡도는 최악의 경우 기준이다.

자료구조	값 20 찾기	찾은 뒤에 삽입	전체
배열	O(N)	O(N)	O(N)
연결 리스트	O(N)	O(1)	O(N)

배열의 O(1) 접근은 인덱스를 알 때 적용된다. 값 20만 알고 위치를 모르면 탐색이 필요하다.

노드 참조가 이미 있다면?

참조 = 특정 노드를 직접 가리키는 정보



ref가 20을 가리키고 있을 때

해당 노드의 값 읽기 **O(1)**

그 바로 뒤에 25 삽입 **O(1)**

단, head부터 찾아서 ref를 얻는 과정은

최악 O(N)이다.

위치 번호 ≠ 노드 참조

k번째라는 번호만 알면 접근 O(k), 최악 O(N).
단, 두 번째처럼 고정된 위치라면 접근도 O(1).